Tropical Cyclone Progression and Wind-Field Model

Bayesian State-Space Progression with Parametric Wind-Field

MKM Research Labs

Version 0.2.0 — 05-June-2026

David K Kelly

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs ("MKM").

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an "as is" basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

Contents

1 Executive Summary

Skeleton document — content to be filled out as the model matures through Phase 1 of the implementation plan (`src/models/typhoon/phase_1_plan.md`).

The Tropical Cyclone Progression and Wind-Field Model (`MKM-TC-001`) simulates the forward motion, intensity evolution, and local wind field of tropical cyclones (typhoons) for catchments exposed to TC hazard. It combines:

- A Bayesian state-space progression engine, advancing a latent storm state through time with regime-conditioned motion, intensity, and size transitions.
- A Sequential Monte Carlo (particle filter) inference layer.
- A parametric radial wind-field with motion-linked asymmetry, surface reduction, and configurable inner-core and outer-decay structure.

The model is catchment-agnostic. Catchment-specific calibration is supplied through `config.typhoon` parameter dataclasses, populated by each catchment's `data/catch/<id>/tc.py` file.

Status: Development. Phase 1.1 (data structures and parameter schema) delivered as of the document date. Phases 1.2 through 1.8 in progress.

Key risk: Phase 1 uses placeholder calibration. The model is not approved for production use until Phase 3 historical calibration completes and the Model Risk Committee (MRC) issues approval.

2 Model Purpose and Scope

2.1 Purpose

To be completed.

2.2 Scope

To be completed.

2.3 Out of Scope

The following are explicitly out of scope for the first production-grade release:

- Full atmospheric data assimilation
- Full dynamical Numerical Weather Prediction
- Joint ocean-atmosphere coupling
- Full pressure-wind structural models
- Operational forecast-grade verification pipelines

3 Mathematical Framework

3.1 Latent State

The latent storm state at time t is

$$s_t = (\lambda_t, \phi_t, u_t, \psi_t, V_t, R_{\max,t}, R_{\text{outer},t}, k_t),$$

where λ, ϕ are longitude and latitude, u is translation speed, ψ is heading, V is maximum sustained wind, $R_{\max}$ and R_{outer} are the radii of maximum wind and the outer (gale-force) wind, and k is a discrete regime class.

3.2 Sequential Bayes Update

To be completed — spec eq. (3) and (4).

3.3 Transition Model

To be completed — spec eqs. (5)–(10).

3.4 Wind-Field Model

To be completed — spec eqs. (22)–(29).

3.5 Storm-Coupled Genesis (Wind-into-PRS)

When invoked as part of the coupled storm–typhoon event set for PRS pricing, genesis is no longer drawn independently. Each typhoon is paired 1:1 with a storm sequence sharing an `event_id`, and the genesis peak wind is *conditioned* on that storm's meteorological severity. The full coupling specification (motivation, calibration, validation criteria) lives in `docs/models/storm_typhoon_coupling/coupling_spec.md`; this subsection records the genesis-layer realisation in `src/models/typhoon/genesis.py`.

Let z be the paired storm's severity latent (`base_intensity`) and $q \in (0, 1)$ its empirical severity quantile across the N-event batch, $q_i = \text{rank}(z_i)/(N+1)$. The event's wind strength percentile ρ_w is drawn within a q-dependent band:

$$\text{floor}(q) = 1 - (1-q)^{\beta}, \qquad \text{ceiling}(q) = q, \tag{1}$$

$$\rho_w = \text{floor}(q) + B\,[\,q - \text{floor}(q)\,], \qquad B \sim \text{Uniform}[0,1], \tag{2}$$

and the genesis peak wind is recovered by inverting the scenario-mix-weighted catchment wind-exceedance curve, $V_{\max} = S_{\text{cat}}^{-1}(1-\rho_w)$ (`coupled_genesis_wind`, using `coupling_floor`, `mixture_peak_wind_excee` and `mixture_peak_wind_inverse`). The coupling is **directed and asymmetric**: wind can never rank above the storm ($\rho_w \le q$, forbidding high-wind/low-rain), but a rising floor pulls the wind up in the storm tail. The single parameter β tunes the strength ($\beta \to 0$ pure ceiling, $\beta = 1$ comonotone; interim $\beta \approx 0.52$ matching expert anchors), shipped as `config.port.COUPLING_BETA` with a `--coupling-beta` override.

Within an event the coupled $V_{\max}$ (and its derived scenario family) **override** the genesis draw for all particles, so the whole SMC ensemble starts from the one coupled intensity while still exploring

track, size, location, and regime; `step()` subsequently evolves $V_{\max}$ so trajectory diversity persists. The typhoon count is slaved to the storm count (`--num-storms`), giving a bijective `event_id` pairing.

Effect on the wind marginal. Because winds are gated below the storm-severity ceiling, the realised marginal distribution of $V_{\max}$ shifts *downward* relative to the standalone (uncoupled) genesis prior: the `tc.py` bands now define the attainable ceiling, not the realised frequency. This is the deliberate cost of "rain without wind, but no wind without rain." Catchments without a `tc.py` produce no wind component (flood-only fallback).

4 Input Parameters and Calibration

To be completed. Parameter schema lives in `config/typhoon.py`; concrete catchment values live in `data/catch/<id>/tc.py`. See parameter inventory section of the governance report for the full enumeration.

5 Implementation

5.1 Module Layout

- `config/typhoon.py` — parameter schema (enums and dataclasses).
- `src/models/typhoon/data_structures.py` — runtime types (`TyphoonState`, `TyphoonParticle`, `TyphoonTrajectory`, `WindFieldOutput`).
- `src/models/typhoon/genesis.py` — genesis prior and tail-aware peak-wind sampling (to be added in Phase 1.2).
- `src/models/typhoon/transitions.py` — one-step state propagator (to be added in Phase 1.3).
- `src/models/typhoon/particle_filter.py` — hand-rolled SMC engine (to be added in Phase 1.4).
- `src/models/typhoon/plausibility.py` — simulation-mode soft constraints (to be added in Phase 1.5).
- `src/models/typhoon/wind_field.py` — parametric radial wind-field (to be added in Phase 1.6).
- `src/models/typhoon/pipeline.py` — end-to-end orchestration (to be added in Phase 1.7).

5.2 Catchment Integration

The model is catchment-agnostic. The invocation path is:

```
MKM_CATCHMENT=<catchment_id> python3 app.py port --typhoon
```

A thin boundary adapter under `app/commands/port/stages/typhoon_stage.py` reads the active catchment's `tc.py` via the config routing layer, assembles a `CatchmentTyphoonConfig` dataclass, and calls `simulate_typhoon_events(...)`. The model never imports from `data/catch/*`.

6 Validation and Backtesting

Phase 3 work — to be completed. Will include:

- Plausible particle-track spread through time, widening with forecast horizon
- Realistic smoothness of heading and speed paths
- Reasonable distribution of recurvature and landfall behaviour by regime
- Stable posterior behaviour under different particle counts and time steps
- Consistency of progression outputs with the downstream wind-field footprint
- Historical validation against named typhoons in the target basin

Table 1: Test Results Summary — Tropical Cyclone Progression and Wind-Field (MKM-TC-001)

Total Tests	271
Passed	271
Failed	0
Skipped	0
Duration	0.36s

Table 2: Detailed Test Results — Tropical Cyclone Progression and Wind-Field

Test Description	**Acceptance Criteria**	**Result**	**Time**
Direct hit shows eye dip	`out.sustained_ms[nearest_idx] < peak`	Pass	0.000s
Direct hit shows eye dip	`out.sustained_ms[nearest_idx] < peak`	Pass	0.000s
Offset passing storm yields unimodal smooth curve	`4 <= peak_idx <= 8; 60.0 < out.distance_at_peak_km < 100.0` (+3 more)	Pass	0.000s
Offset passing storm yields unimodal smooth curve	`4 <= peak_idx <= 8; 60.0 < out.distance_at_peak_km < 100.0` (+3 more)	Pass	0.000s
Empty realizations	`s.n_realizations == 0; s.peak_sustained_samples == []` (+1 more)	Pass	0.000s
Empty realizations	`s.n_realizations == 0; s.peak_sustained_samples == []` (+1 more)	Pass	0.000s
Mean and quantiles	`s.n_realizations == 5; s.peak_sustained_mean == pytest.approx(30.0)` (+3 more)	Pass	0.001s

Test Description	Acceptance Criteria	Result	Time
Mean and quantiles	`s.n_realizations == 5`; `s.peak_sustained_mean == pytest.approx(30.0)` (+3 more)	Pass	0.000s
Threshold exceedance	`s.threshold_exceedance[17.5] == 0.5`; `s.mean_duration_above_ms[17.5] == pytest.approx(1.5)`	Pass	0.000s
Threshold exceedance	`s.threshold_exceedance[17.5] == 0.5`; `s.mean_duration_above_ms[17.5] == pytest.approx(1.5)`	Pass	0.000s
Build typhoon config accepts explicit catchment id	`cfg.catchment_id == "halong"`	Pass	0.000s
Build typhoon config returns valid assembly	`cfg.catchment_id == "halong"`; `cfg.land_mask(ref_lon, ref_lat) is True` (+2 more)	Pass	0.000s
Enhancement along phi	`asymmetry_factor(45.0, 45.0, 0.2) == pytest.approx(1.2, a...`	Pass	0.000s
Enhancement along phi	`asymmetry_factor(45.0, 45.0, 0.2) == pytest.approx(1.2, a...`	Pass	0.000s
Neutral when eps zero	`asymmetry_factor(0.0, 0.0, 0.0) == 1.0`	Pass	0.000s
Neutral when eps zero	`asymmetry_factor(0.0, 0.0, 0.0) == 1.0`	Pass	0.000s
Suppression opposite phi	`asymmetry_factor(225.0, 45.0, 0.2) == pytest.approx(0.8, ...`	Pass	0.000s
Suppression opposite phi	`asymmetry_factor(225.0, 45.0, 0.2) == pytest.approx(0.8, ...`	Pass	0.000s
Due east is 90	`b == pytest.approx(90.0, abs=0.01)`	Pass	0.000s
Due east is 90	`b == pytest.approx(90.0, abs=0.01)`	Pass	0.000s
Due north is 0	`b == pytest.approx(0.0, abs=0.01)`	Pass	0.000s
Due north is 0	`b == pytest.approx(0.0, abs=0.01)`	Pass	0.000s
Due south is 180	`b == pytest.approx(180.0, abs=0.01)`	Pass	0.000s
Due south is 180	`b == pytest.approx(180.0, abs=0.01)`	Pass	0.000s
Due west is 270	`b == pytest.approx(270.0, abs=0.01)`	Pass	0.000s
Due west is 270	`b == pytest.approx(270.0, abs=0.01)`	Pass	0.000s
In range	`0.0 <= b < 360.0`	Pass	0.000s
In range	`0.0 <= b < 360.0`	Pass	0.000s
Anchors v sym at r outer to reference	`v_at_outer == pytest.approx(v_ref, rel=1e-6)`	Pass	0.000s
Anchors v sym at r outer to reference	`v_at_outer == pytest.approx(v_ref, rel=1e-6)`	Pass	0.000s
Fallback when geometry invalid	`L == 1.0`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Fallback when geometry invalid	`L == 1.0`	Pass	0.000s
Fallback when v max below reference	`L == pytest.approx(120.0, abs=1e-9)`	Pass	0.000s
Fallback when v max below reference	`L == pytest.approx(120.0, abs=1e-9)`	Pass	0.000s
Empty	`duration_above_threshold([], [], threshold=5.0) == 0.0`	Pass	0.000s
Empty	`duration_above_threshold([], [], threshold=5.0) == 0.0`	Pass	0.000s
Full when all above	`duration_above_threshold([0.0, 1.0, 2.0], [50.0, 50.0, 50...`	Pass	0.000s
Full when all above	`duration_above_threshold([0.0, 1.0, 2.0], [50.0, 50.0, 50...`	Pass	0.000s
Partial count	`duration_above_threshold(ts, vs, threshold=10.0) == 3.0`	Pass	0.000s
Partial count	`duration_above_threshold(ts, vs, threshold=10.0) == 3.0`	Pass	0.000s
Zero when all below	`duration_above_threshold([0.0, 1.0, 2.0], [5.0, 5.0, 5.0]...`	Pass	0.000s
Zero when all below	`duration_above_threshold([0.0, 1.0, 2.0], [5.0, 5.0, 5.0]...`	Pass	0.000s
Capped at eps max	`eps == 0.3`	Pass	0.000s
Capped at eps max	`eps == 0.3`	Pass	0.000s
Typical storm	`eps == pytest.approx(0.6 * (20.0 / 3.6) / 41.0, abs=1e-9)`	Pass	0.000s
Typical storm	`eps == pytest.approx(0.6 * (20.0 / 3.6) / 41.0, abs=1e-9)`	Pass	0.000s
Zero translation gives zero	`eps == pytest.approx(0.0, abs=1e-9)`	Pass	0.000s
Zero translation gives zero	`eps == pytest.approx(0.0, abs=1e-9)`	Pass	0.000s
At storm center value is alpha eye with sea	`v == pytest.approx(default_params.alpha_eye * 40.0, abs=1...`	Pass	0.000s
At storm center value is alpha eye with sea	`v == pytest.approx(default_params.alpha_eye * 40.0, abs=1...`	Pass	0.000s
Doubling v max doubles evaluation in inner core	`v2 == pytest.approx(2.0 * v1, rel=1e-9)`	Pass	0.000s
Doubling v max doubles evaluation in inner core	`v2 == pytest.approx(2.0 * v1, rel=1e-9)`	Pass	0.000s
Faster storm enhances nh right side	`v_fast > v_slow`	Pass	0.000s
Faster storm enhances nh right side	`v_fast > v_slow`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Geometry returned alongside wind	`isinstance(result, PointWind)`; `result.distance_km > 0.0` (+2 more)	Pass	0.000s
Geometry returned alongside wind	`isinstance(result, PointWind)`; `result.distance_km > 0.0` (+2 more)	Pass	0.000s
Land reduction applied	`v_land == pytest.approx(expected, rel=1e-6)`	Pass	0.000s
Land reduction applied	`v_land == pytest.approx(expected, rel=1e-6)`	Pass	0.000s
Empty trajectory	`out.time_hours == []`; `out.peak_sustained_ms == 0.0`	Pass	0.000s
Empty trajectory	`out.time_hours == []`; `out.peak_sustained_ms == 0.0`	Pass	0.000s
Peak metadata matches peak sample	`out.time_of_peak_hours == out.time_hours[peak_idx]`	Pass	0.000s
Peak metadata matches peak sample	`out.time_of_peak_hours == out.time_hours[peak_idx]`	Pass	0.000s
Returns wind field output shape	`out.point_id == "P1"`; `len(out.time_hours) == len(traj.states)` (+2 more)	Pass	0.000s
Returns wind field output shape	`out.point_id == "P1"`; `len(out.time_hours) == len(traj.states)` (+2 more)	Pass	0.000s
Bbox inside scs	`105.0 <= lon_min < lon_max <= 125.0`; `5.0 <= lat_min < lat_max <= 25.0`	Pass	0.000s
Is genesis prior	`isinstance(halong_tc.HALONG_TYPHOON_GENESIS_PRIOR, Genesi...`	Pass	0.000s
Regime weights sum to one	`set(weights.keys()) == set(RegimeClass)`; `math.isclose(sum(weights.values()), 1.0, abs_tol=1e-9)`	Pass	0.000s
Scenario mix sums to one	`set(mix.keys()) == set(ScenarioFamily)`; `math.isclose(sum(mix.values()), 1.0, abs_tol=1e-9)`	Pass	0.000s
Bbox ordering	`lon_min < lon_max`; `lat_min < lat_max`	Pass	0.000s
Empty weights acceptable at construction	`g.regime_weights == {}`; `g.scenario_mix == {}`	Pass	0.000s
One degree latitude is pi r over 180	`d == pytest.approx(expected, abs=1e-3)`	Pass	0.000s
One degree latitude is pi r over 180	`d == pytest.approx(expected, abs=1e-3)`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Symmetric	`d_ab == pytest.approx(d_ba, abs=1e-9)`	Pass	0.000s
Symmetric	`d_ab == pytest.approx(d_ba, abs=1e-9)`	Pass	0.000s
Zero for coincident points	`haversine_distance_km(LON, LAT, LON, LAT) == pytest.appro...`	Pass	0.000s
Zero for coincident points	`haversine_distance_km(LON, LAT, LON, LAT) == pytest.appro...`	Pass	0.000s
At least one source file exists	`len(files) >= 1`	Pass	0.003s
At least one source file exists	`len(files) >= 1`	Pass	0.002s
Top-level imports should come from stdlib, well-known third-party	`not unknown`	Pass	0.018s
Top-level imports should come from stdlib, well-known third-party	`not unknown`	Pass	0.018s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='HALONG']	`not offenders`	Pass	0.006s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='HALONG']	`not offenders`	Pass	0.008s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='HANOI']	`not offenders`	Pass	0.006s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='HANOI']	`not offenders`	Pass	0.006s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='Halong']	`not offenders`	Pass	0.004s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='Halong']	`not offenders`	Pass	0.005s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='Hanoi']	`not offenders`	Pass	0.004s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='Hanoi']	`not offenders`	Pass	0.005s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='THAMES']	`not offenders`	Pass	0.005s

Test Description	Acceptance Criteria	Result	Time
No catchment name may appear in src/models/typhoon/ source. [catchment_name='THAMES']	`not offenders`	Pass	0.005s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='Thames']	`not offenders`	Pass	0.005s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='Thames']	`not offenders`	Pass	0.007s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='halong']	`not offenders`	Pass	0.005s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='halong']	`not offenders`	Pass	0.004s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='hanoi']	`not offenders`	Pass	0.006s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='hanoi']	`not offenders`	Pass	0.005s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='thames']	`not offenders`	Pass	0.011s
No catchment name may appear in src/models/typhoon/ source. [catchment_name='thames']	`not offenders`	Pass	0.004s
No source file imports forbidden package[app] [forbidden='app']	`not offenders`	Pass	0.049s
No source file imports forbidden package[app] [forbidden='app']	`not offenders`	Pass	0.019s
No source file imports forbidden package[catch] [forbidden='catch']	`not offenders`	Pass	0.020s
No source file imports forbidden package[catch] [forbidden='catch']	`not offenders`	Pass	0.018s
No source file imports forbidden package[port] [forbidden='port']	`not offenders`	Pass	0.020s
No source file imports forbidden package[port] [forbidden='port']	`not offenders`	Pass	0.018s
Within the config package, only config.typhoon (the parameter	`not offenders`	Pass	0.020s

Test Description	Acceptance Criteria	Result	Time
Within the config package, only config.typhoon (the parameter	`not offenders`	Pass	0.019s
Construct with defaults	`p.k_land_per_hour > 0;` `p.sigma_ms_per_hour > 0` (+1 more)	Pass	0.000s
Drift can be zero	`p.drift_ms_per_hour == 0.0`	Pass	0.000s
Intensity	`isinstance(halong_tc.HALONG_TYPHOON_INTENSITY, IntensityP...`	Pass	0.000s
Plausibility	`isinstance(halong_tc.HALONG_TYPHOON_PLAUSIBILITY, Plausib...`	Pass	0.000s
Size	`isinstance(halong_tc.HALONG_TYPHOON_SIZE, SizeParams)`	Pass	0.000s
Wind field	`isinstance(halong_tc.HALONG_TYPHOON_WIND_FIELD, WindField...`	Pass	0.000s
Gulf of tonkin is sea	`halong_tc.halong_land_mask(108.0, 20.0) is False`	Pass	0.000s
Hainan is land	`halong_tc.halong_land_mask(109.5, 19.0) is True`	Pass	0.000s
Hanoi is land	`halong_tc.halong_land_mask(lon, lat) is True`	Pass	0.000s
Open scs is sea	`halong_tc.halong_land_mask(115.0, 18.0) is False`	Pass	0.000s
All regimes covered	`r in m.mean_speed_kmh; r in m.sigma_speed_kmh` (+2 more)	Pass	0.000s
Is motion params	`isinstance(halong_tc.HALONG_TYPHOON_MOTION, MotionParams)`	Pass	0.000s
Stalled regime has lowest mean speed	`speeds[RegimeClass.STALLED] == min(speeds.values())`	Pass	0.000s
All regimes covered in motion	`r in neutral_config.motion.mean_speed_kmh; r in neutral_config.motion.mean_heading_deg`	Pass	0.000s
All scenario families covered	`s in neutral_config.peak_wind`	Pass	0.000s
Construction	`isinstance(neutral_config, CatchmentTyphoonConfig); neutral_config.catchment_id == "test"`	Pass	0.000s
Default horizon is one week	`neutral_config.horizon_hours == 168.0`	Pass	0.000s
Default output thresholds ascending	`thresholds == sorted(thresholds); all(t > 0 for t in thresholds)`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Land mask is callable	`callable(neutral_config.land_mask); neutral_config.land_mask(115.0, 21.0) is True` (+1 more)	Pass	0.000s
Property points non empty	`len(neutral_config.property_points) >= 1`	Pass	0.000s
Covers all scenarios	`set(pw.keys()) == set(ScenarioFamily); isinstance(params, PeakWindParams)`	Pass	0.000s
Extreme has fatter tail than baseline	`pw[ScenarioFamily.EXTREME].alpha < pw[ScenarioFamily.BASE...`	Pass	0.000s
Means weakly increase with severity	`means == sorted(means)`	Pass	0.000s
Fatter tail smaller alpha	`fat.alpha < thin.alpha`	Pass	0.000s
Simple construction	`p.mu_ms == 40.0; p.v_min_ms < p.mu_ms < p.v_max_ms`	Pass	0.000s
Motion plus offset wraps	`compute_phi_deg(270.0, 90.0) == 0.0`	Pass	0.000s
Motion plus offset wraps	`compute_phi_deg(270.0, 90.0) == 0.0`	Pass	0.000s
Wrapping handled	`compute_phi_deg(350.0, 30.0) == 20.0`	Pass	0.000s
Wrapping handled	`compute_phi_deg(350.0, 30.0) == 20.0`	Pass	0.000s
Empty input returns none	`pick_representative_trajectory([]) is None`	Pass	0.000s
Empty input returns none	`pick_representative_trajectory([]) is None`	Pass	0.000s
Returns median peak trajectory	`rep is not None; rep in result.trajectories`	Pass	0.001s
Returns median peak trajectory	`rep is not None; rep in result.trajectories`	Pass	0.001s
All weights loose	`p.heading_jump_weight <= 0.5; p.speed_jump_weight <= 0.5` (+2 more)	Pass	0.000s
Centre matches reference	`centre.longitude == ref_lon; centre.latitude == ref_lat`	Pass	0.000s
Each is property point	`isinstance(p, PropertyPoint)`	Pass	0.000s
Ids unique	`len(ids) == len(set(ids))`	Pass	0.000s
Non empty	`len(halong_tc.HALONG_TYPHOON_PROPERTIES) >= 1`	Pass	0.000s
From dict recovers float threshold keys	`isinstance(k, float)`	Pass	0.000s
From dict recovers float threshold keys	`isinstance(k, float)`	Pass	0.000s
Roundtrip	`s2 == s; ens2 == ens` (+1 more)	Pass	0.000s
Roundtrip	`s2 == s; ens2 == ens` (+1 more)	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
To dict encodes threshold keys as strings	`isinstance(k, str); isinstance(k, str)`	Pass	0.000s
To dict encodes threshold keys as strings	`isinstance(k, str); isinstance(k, str)`	Pass	0.000s
Five regimes exist	`len(RegimeClass) == 5`	Pass	0.000s
Five regimes exist	`len(RegimeClass) == 5`	Pass	0.000s
Five regimes exist	`len(RegimeClass) == 5`	Pass	0.000s
Landfall decay value	`RegimeClass.LANDFALL_DECAY.value == "landfall_decay"`	Pass	0.000s
Landfall decay value	`RegimeClass.LANDFALL_DECAY.value == "landfall_decay"`	Pass	0.000s
Nw recurver value	`RegimeClass.NW_RECURVER.value == "nw_recurver"`	Pass	0.000s
Nw recurver value	`RegimeClass.NW_RECURVER.value == "nw_recurver"`	Pass	0.000s
Roundtrip via value	`RegimeClass(r.value) is r; ScenarioFamily(s.value) is s`	Pass	0.000s
Roundtrip via value	`RegimeClass(r.value) is r; ScenarioFamily(s.value) is s`	Pass	0.000s
Roundtrip via value[regimeclass.landfall decay] [regime=¡RegimeClass.LANDFALL_DECAY: ’landfall_decay’¿]	`RegimeClass(regime.value) is regime; ScenarioFamily(scenario.value) is scenario`	Pass	0.000s
Roundtrip via value[regimeclass.nw recurver] [regime=¡RegimeClass.NW_RECURVER: ’nw_recurver’¿]	`RegimeClass(regime.value) is regime; ScenarioFamily(scenario.value) is scenario`	Pass	0.000s
Roundtrip via value[regimeclass.sharp recurve] [regime=¡RegimeClass.SHARP_RECURVE: ’sharp_recurve’¿]	`RegimeClass(regime.value) is regime; ScenarioFamily(scenario.value) is scenario`	Pass	0.000s
Roundtrip via value[regimeclass.stalled] [regime=¡RegimeClass.STALLED: ’stalled’¿]	`RegimeClass(regime.value) is regime; ScenarioFamily(scenario.value) is scenario`	Pass	0.000s
Roundtrip via value[regimeclass.straight westward] [regime=¡RegimeClass.STRAIGHT_WESTWARD: ’straight_westward’¿]	`RegimeClass(regime.value) is regime; ScenarioFamily(scenario.value) is scenario`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Sharp recurve value	`RegimeClass.SHARP_RECURVE.value == "sharp_recurve"`	Pass	0.000s
Sharp recurve value	`RegimeClass.SHARP_RECURVE.value == "sharp_recurve"`	Pass	0.000s
Stalled value	`RegimeClass.STALLED.value == "stalled"`	Pass	0.000s
Stalled value	`RegimeClass.STALLED.value == "stalled"`	Pass	0.000s
Straight westward value	`RegimeClass.STRAIGHT_WESTWARD.value == "straight_westward"`	Pass	0.000s
Straight westward value	`RegimeClass.STRAIGHT_WESTWARD.value == "straight_westward"`	Pass	0.000s
Values are lower snake case	`r.value == r.value.lower(); " " not in r.value`	Pass	0.000s
Baseline value	`ScenarioFamily.BASELINE.value == "baseline"`	Pass	0.000s
Baseline value	`ScenarioFamily.BASELINE.value == "baseline"`	Pass	0.000s
Extreme value	`ScenarioFamily.EXTREME.value == "extreme"`	Pass	0.000s
Extreme value	`ScenarioFamily.EXTREME.value == "extreme"`	Pass	0.000s
Five families exist	`len(ScenarioFamily) == 5`	Pass	0.000s
Five families exist	`len(ScenarioFamily) == 5`	Pass	0.000s
Five families exist	`len(ScenarioFamily) == 5`	Pass	0.000s
Historical value	`ScenarioFamily.HISTORICAL.value == "historical"`	Pass	0.000s
Historical value	`ScenarioFamily.HISTORICAL.value == "historical"`	Pass	0.000s
Moderate value	`ScenarioFamily.MODERATE.value == "moderate"`	Pass	0.000s
Moderate value	`ScenarioFamily.MODERATE.value == "moderate"`	Pass	0.000s
Roundtrip via value	`RegimeClass(r.value) is r; ScenarioFamily(s.value) is s`	Pass	0.000s
Roundtrip via value	`RegimeClass(r.value) is r; ScenarioFamily(s.value) is s`	Pass	0.000s
Roundtrip via value[scenariofamily.baseline] [scenario=¡ScenarioFamily.BASELINE: ’baseline’¿]	`RegimeClass(regime.value) is regime; ScenarioFamily(scenario.value) is scenario`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Roundtrip via value[scenariofamily.extreme] [scenario=¡ScenarioFamily.EXTREME: 'extreme'¿]	`RegimeClass(regime.value) is regime; ScenarioFamily(scenario.value) is scenario`	Pass	0.000s
Roundtrip via value[scenariofamily.historical] [scenario=¡ScenarioFamily.HISTORICAL: 'historical'¿]	`RegimeClass(regime.value) is regime; ScenarioFamily(scenario.value) is scenario`	Pass	0.000s
Roundtrip via value[scenariofamily.moderate] [scenario=¡ScenarioFamily.MODERATE: 'moderate'¿]	`RegimeClass(regime.value) is regime; ScenarioFamily(scenario.value) is scenario`	Pass	0.000s
Roundtrip via value[scenariofamily.severe] [scenario=¡ScenarioFamily.SEVERE: 'severe'¿]	`RegimeClass(regime.value) is regime; ScenarioFamily(scenario.value) is scenario`	Pass	0.000s
Severe value	`ScenarioFamily.SEVERE.value == "severe"`	Pass	0.000s
Severe value	`ScenarioFamily.SEVERE.value == "severe"`	Pass	0.000s
Fixed seed reproducible	`peaks_a == peaks_b`	Pass	0.001s
Fixed seed reproducible	`peaks_a == peaks_b`	Pass	0.001s
List length matches n particles	`len(result.trajectories) == 15; len(outputs) == 15`	Pass	0.001s
List length matches n particles	`len(result.trajectories) == 15; len(outputs) == 15`	Pass	0.001s
No plausibility path	`len(outputs) == 5`	Pass	0.000s
No plausibility path	`len(outputs) == 5`	Pass	0.000s
Outputs are wind field output instances	`isinstance(t, TyphoonTrajectory); isinstance(o, WindFieldOutput)`	Pass	0.001s
Outputs are wind field output instances	`isinstance(t, TyphoonTrajectory); isinstance(o, WindFieldOutput)`	Pass	0.001s
Returns event result	`isinstance(result, EventResult); len(result.trajectories) == 10` (+1 more)	Pass	0.004s
Returns event result	`isinstance(result, EventResult); len(result.trajectories) == 10` (+1 more)	Pass	0.001s
Construct with defaults	`p.k_land_per_hour > 0; p.sigma_ms_per_hour > 0` (+1 more)	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
R outer intercept larger than r max	`math.exp(p.r_outer_intercept_log_km) > math.exp(p.r_max_i...`	Pass	0.000s
Land uses rho surf land	`surface_factor(is_land=True, rho_surf_sea=1.0, rho_surf_l...`	Pass	0.000s
Land uses rho surf land	`surface_factor(is_land=True, rho_surf_sea=1.0, rho_surf_l...`	Pass	0.000s
Sea uses rho surf sea	`surface_factor(is_land=False, rho_surf_sea=1.0, rho_surf_...`	Pass	0.000s
Sea uses rho surf sea	`surface_factor(is_land=False, rho_surf_sea=1.0, rho_surf_...`	Pass	0.000s
Doubling v max doubles v sym at fixed geometry	`v2 == pytest.approx(2.0 * v1, abs=1e-9)`	Pass	0.000s
Doubling v max doubles v sym at fixed geometry	`v2 == pytest.approx(2.0 * v1, abs=1e-9)`	Pass	0.000s
Inner core is linear ramp	`v == pytest.approx(expected, abs=1e-9)`	Pass	0.000s
Inner core is linear ramp	`v == pytest.approx(expected, abs=1e-9)`	Pass	0.000s
Outer decays below v max	`v < 40.0`; `v > 0.0`	Pass	0.000s
Outer decays below v max	`v < 40.0`; `v > 0.0`	Pass	0.000s
Value at center is alpha eye times v max	`v == pytest.approx(0.4 * 40.0, abs=1e-9)`	Pass	0.000s
Value at center is alpha eye times v max	`v == pytest.approx(0.4 * 40.0, abs=1e-9)`	Pass	0.000s
Value at r max is v max	`v == pytest.approx(40.0, abs=1e-9)`	Pass	0.000s
Value at r max is v max	`v == pytest.approx(40.0, abs=1e-9)`	Pass	0.000s
Empty properties acceptable	`ens2.properties == []`	Pass	0.000s
Empty properties acceptable	`ens2.properties == []`	Pass	0.000s
Roundtrip	`s2 == s`; `ens2 == ens` (+1 more)	Pass	0.000s
Roundtrip	`s2 == s`; `ens2 == ens` (+1 more)	Pass	0.000s
Construction	`sample_state.longitude == 5.0`; `sample_state.regime is RegimeClass.STRAIGHT_WESTWARD` (+10 more)	Pass	0.000s
Construction	`sample_state.longitude == 5.0`; `sample_state.regime is RegimeClass.STRAIGHT_WESTWARD` (+10 more)	Pass	0.000s
Genesis particle has no parent	`p.parent_id is None`	Pass	0.000s
Genesis particle has no parent	`p.parent_id is None`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Roundtrip	`restored == sample_state; restored == sample_particle` (+2 more)	Pass	0.000s
Roundtrip	`restored == sample_state; restored == sample_particle` (+2 more)	Pass	0.000s
Roundtrip with none parent	`restored == p; restored.parent_id is None`	Pass	0.000s
Roundtrip with none parent	`restored == p; restored.parent_id is None`	Pass	0.000s
Construction	`sample_state.longitude == 5.0; sample_state.regime is RegimeClass.STRAIGHT_WESTWARD` (+10 more)	Pass	0.000s
Construction	`sample_state.longitude == 5.0; sample_state.regime is RegimeClass.STRAIGHT_WESTWARD` (+10 more)	Pass	0.000s
Roundtrip	`restored == sample_state; restored == sample_particle` (+2 more)	Pass	0.000s
Roundtrip	`restored == sample_state; restored == sample_particle` (+2 more)	Pass	0.000s
Roundtrip landfall state	`restored == landfall_state; restored.land_flag is True` (+1 more)	Pass	0.000s
Roundtrip landfall state	`restored == landfall_state; restored.land_flag is True` (+1 more)	Pass	0.000s
To dict contains all fields	`set(d.keys()) == expected_keys`	Pass	0.000s
To dict contains all fields	`set(d.keys()) == expected_keys`	Pass	0.000s
To dict encodes regime as string	`d["regime"] == "straight_westward"`	Pass	0.000s
To dict encodes regime as string	`d["regime"] == "straight_westward"`	Pass	0.000s
Construction	`sample_state.longitude == 5.0; sample_state.regime is RegimeClass.STRAIGHT_WESTWARD` (+10 more)	Pass	0.000s
Construction	`sample_state.longitude == 5.0; sample_state.regime is RegimeClass.STRAIGHT_WESTWARD` (+10 more)	Pass	0.000s
Empty trajectory horizon is zero	`t.n_steps == 0; t.horizon_hours == 0.0`	Pass	0.000s
Empty trajectory horizon is zero	`t.n_steps == 0; t.horizon_hours == 0.0`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Horizon hours matches final state	`sample_trajectory.horizon_hours == 72.0`	Pass	0.000s
Horizon hours matches final state	`sample_trajectory.horizon_hours == 72.0`	Pass	0.000s
Roundtrip	`restored == sample_state; restored == sample_particle` (+2 more)	Pass	0.000s
Roundtrip	`restored == sample_state; restored == sample_particle` (+2 more)	Pass	0.000s
Constructs from config	`wf.config is minimal_config; wf.params is minimal_config.wind_field` (+1 more)	Pass	0.000s
Constructs from config	`wf.config is minimal_config; wf.params is minimal_config.wind_field` (+1 more)	Pass	0.000s
Evaluate matches functional	`class_call == pytest.approx(functional, abs=1e-12)`	Pass	0.000s
Evaluate matches functional	`class_call == pytest.approx(functional, abs=1e-12)`	Pass	0.000s
Evaluate property passes lon lat	`a == pytest.approx(b, abs=1e-12)`	Pass	0.000s
Evaluate property passes lon lat	`a == pytest.approx(b, abs=1e-12)`	Pass	0.000s
Evaluate trajectory uses config thresholds by default	`set(out.duration_above_ms.keys()) == set(minimal_config.o...`	Pass	0.000s
Evaluate trajectory uses config thresholds by default	`set(out.duration_above_ms.keys()) == set(minimal_config.o...`	Pass	0.000s
Explicit thresholds override default	`set(out.duration_above_ms.keys()) == {20.0, 30.0}`	Pass	0.000s
Explicit thresholds override default	`set(out.duration_above_ms.keys()) == {20.0, 30.0}`	Pass	0.000s
Construction	`sample_state.longitude == 5.0; sample_state.regime is RegimeClass.STRAIGHT_WESTWARD` (+10 more)	Pass	0.000s
Construction	`sample_state.longitude == 5.0; sample_state.regime is RegimeClass.STRAIGHT_WESTWARD` (+10 more)	Pass	0.000s
Peak matches max of series	`sample_wind_output.peak_sustained_ms == max(sample_wind_o...`	Pass	0.000s
Peak matches max of series	`sample_wind_output.peak_sustained_ms == max(sample_wind_o...`	Pass	0.000s
Roundtrip	`restored == sample_state; restored == sample_particle` (+2 more)	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Roundtrip	`restored == sample_state; restored == sample_particle` (+2 more)	Pass	0.000s
Roundtrip recovers float threshold keys	`isinstance(k, float)`	Pass	0.000s
Roundtrip recovers float threshold keys	`isinstance(k, float)`	Pass	0.000s
To dict encodes threshold keys as strings	`isinstance(k, str)`	Pass	0.000s
To dict encodes threshold keys as strings	`isinstance(k, str)`	Pass	0.000s
Alpha eye is fractional	`0.0 < p.alpha_eye < 1.0`	Pass	0.000s
Land reduction no stronger than sea	`0.0 <= p.rho_surf_land <= p.rho_surf_sea`	Pass	0.000s
Outer anchor below typical v max	`p.v_outer_ref_ms < 30.0`	Pass	0.000s
Outer shape positive	`p.outer_shape_p > 0`	Pass	0.000s

6.1 Test Document History

Date	Version	Description	Author
27-May-2026	1.0	Initial test suite (150 tests)	D. Kelly
27-May-2026	1.1	36 test(s) added (total: 222)	D. Kelly
27-May-2026	1.2	54 test(s) added (total: 330)	D. Kelly
27-May-2026	1.3	43 test(s) added (total: 416)	D. Kelly
27-May-2026	1.4	33 test(s) added (total: 482)	D. Kelly
27-May-2026	1.5	45 test(s) added (total: 572)	D. Kelly
27-May-2026	1.6	24 test(s) added; 1 test(s) removed (total: 617)	D. Kelly
03-Jun-2026	1.7	10 test(s) added; 1 test(s) removed (total: 346)	D. Kelly
15-Jun-2026	1.8	34 test(s) added (total: 380)	D. Kelly
21-Jul-2026	1.9	216 test(s) removed (total: 164)	D. Kelly

7 Sensitivity Analysis

To be completed in Phase 1.7+. Sensitivities will be generated by ***docs/models/sensitivities/typhoon/generat***

8 Limitations and Assumptions

Refer to entry `MKM-TC-001` in `data/model_inventory.json` for the canonical list. Phase 1 limitations include:

- No historical calibration — parameters are first-pass placeholders.
- Land mask is a simple bbox placeholder rather than a true coastline polygon.

- No real-observation assimilation in Phase 1 (simulation mode only).
- Joint surge/rain hazard layer deferred to Phase 2.
- In storm-coupled mode (Section 3.5) the wind marginal is conditional and shifts downward versus the standalone prior; the coupling strength β is a provisional expert prior pending calibration.

9 Change History

Version	Date	Description
0.1.0	27-May-2026	Skeleton document. Phase 1.1 delivered: parameter schema, runtime types, launch-cat
0.2.0	05-Jun-2026	Added Section 3.5 Storm-Coupled Genesis: severity-quantile band draw (`coupled_gen`